Self-Distilling Rewards from Environment Feedback

Research Proposal — NeurIPS 2026

Rohan Sikand · Stanford University

March 2026

Where Should Reward Signals Come From?

A central question in RL post-training: **how do you tell the model what's good?**

Paradigm	**Signal source**	**Adapts?**	**Failure mode**
Learned RM (RLHF)	Human preferences	Static	Reward hacking
External LLM judge	Prompted evaluation	Static	Distributional gap
Self-reward	Policy's own judgment	On-policy	Reward collapse
Ours	**Policy + env feedback**	**On-policy**	**Grounded**

Key insight: Environments already produce rich structured feedback (stack traces, errors, partial results). The policy can interpret this feedback to judge its own outputs — grounded, adaptive, and cheap.

The General Framework

In any RL post-training loop, the environment produces **more information than is used for reward**. We propose a general mechanism:

Two roles, same model:

$$\underbrace{\pi_\theta(\cdot \mid s)}_{\text{Student}}$$

$$\underbrace{\pi_\theta(\cdot \mid s, c_i, p_{\text{judge}})}_{\text{Teacher}}$$

where c_i = rollout + environment feedback

Three properties:

- **On-policy:** parameter-aligned, zero capability gap
- **Grounded:** env feedback prevents collapse
- **Dense:** structured judgment $\rightarrow$ continuous reward

The framework instantiates wherever there is structured environment feedback — the grounding source changes, the mechanism doesn't.

The Environment Feedback is Already There

Domain	Environment feedback	Currently used as
Code generation	Stack traces, compiler errors, test outputs	Binary pass/fail
Math reasoning	Interpreter errors, assertion failures	Binary correct/wrong
Tool use / agents	API errors, timeout logs, partial outputs	Binary success/fail
Formal verification	Proof checker messages, counterexamples	Binary valid/invalid
Summarization	NLI scores vs source	Scalar threshold
Instruction following	Format/constraint checkers	Binary compliance

In every case, rich structured feedback is **produced and discarded** after computing a coarse reward. The policy itself can interpret this feedback.

RLVR Instantiation: Why It Matters Most Here

RLVR (GRPO) is the cleanest instantiation because the bottleneck is most acute:

GRPO advantages: $\hat{A}_i = (r_i - \mu)/\sigma$

Binary rewards create two failure modes:

1. **All pass or all fail** $\rightarrow \sigma = 0 \rightarrow$ zero gradient signal
2. **No credit assignment** $\rightarrow$ 9/10 correct $=$ same reward as random

And the environment feedback is richest here — stack traces, compiler errors, partial test outputs provide detailed diagnostic information that the model can already interpret.

We instantiate the framework as **EGSR** (Environment-Grounded Self-Reward) and validate on code generation and mathematical reasoning.

Dense Self-Reward Extraction

Teacher outputs judgment over structured classes $\mathcal{K}$:

$$R_{\text{self}}(s, a_i) = \sum_{k \in \mathcal{K}} w_k \cdot P_\theta(q = k \mid s, c_i, p_{\text{judge}})$$

Class	Weight
`correct`	$+1.0$
`minor_bug`	$+0.6$
`wrong_approach`	$+0.2$
`critical_error`	0.0
`exploit`	-0.5

Continuous reward in $[-0.5, 1.0]$ from a single forward pass. Domain-specific: judgment classes change per domain, mechanism doesn't.

Why Grounding is the Key Ingredient

Three functions of environment feedback as privileged information:

1. Objectivity — feedback is produced by the environment, not the model. Stack traces don't lie. Unlike ground-truth solutions (OPSD/SDPO), this is external and unforgeable.

2. Collapse prevention — without f_i, self-reward degrades to sycophancy. With f_i, `IndexError at line 4` is an inescapable constraint.

3. Information recovery:

$$I(Q^*; f \mid R_{\text{env}}, a) = H(Q^* \mid R_{\text{env}}, a) - H(Q^* \mid R_{\text{env}}, a, f) > 0$$

A failing rollout could be nearly correct or total nonsense — feedback resolves this ambiguity.

Label-Free Judge Maintenance

Problem: RL optimizes θ for generation, not judgment. Nothing explicitly trains the teacher.

Two self-supervised objectives from signals already in the loop:

1. **Feedback prediction** — predict f_i from (s, a_i):

$$\mathcal{L}_{\text{pred}}(\theta) = -\mathbb{E}\left[\frac{1}{|f_i|}\sum_k \log \pi_\theta(f_{i,k} \mid s, a_i, f_{i,<k})\right]$$

{Forces internalization of input $\rightarrow$ feedback causality. Always on-distribution.}

2. **Verifier calibration** — anchor R_{self} to ground truth:

$$\mathcal{L}_{\text{cal}}(\theta) = \mathbb{E}\Big[(R_{\text{self}}(s, a_i) - R_{\text{env}}(s, a_i))^2\Big]$$

Joint: $\theta \leftarrow \theta - \eta\, \nabla_\theta[\mathcal{L}_{\text{GRPO}} + \lambda_1\, \mathcal{L}_{\text{pred}} + \lambda_2\, \mathcal{L}_{\text{cal}}]$

Multiple Benefits from One Mechanism

The self-distilling reward framework yields several downstream benefits simultaneously:

Dense reward

- Partial credit for near-correct solutions
- Non-zero gradient in all-pass/all-fail regimes
- Failure-mode-specific shaping

On-policy adaptation

- No distributional gap (parameter-aligned)
- Teacher evolves with student
- No evasion possible

Reward hacking suppression

- Explicit exploit class ($w = -0.5$)
- Grounded: teacher reads the evidence
- $\mathcal{L}_{\text{pred}}$ constrains internal model

Compute efficiency

- Denser signal per rollout
- Smaller group size G needed
- One forward pass, no external model

Theoretical Motivation

Proposition 1: Gradient variance reduction

Binary: $\mathrm{Var}[\hat{A}] \propto p(1-p)/G \rightarrow$ vanishes at extreme pass rates

Dense: $\mathrm{Var}[\hat{A}] \geq \sigma_R^2/G > 0$ regardless of p

Proposition 2: Information-theoretic bound

$I(Q^*; f \mid R_{\mathrm{env}}, a) > 0$ — upper bounds improvement from leveraging feedback

Proposition 3: Calibration safety

If $\mathcal{L}_{\mathrm{cal}} = 0$: optimal policies under $R_{\mathrm{self}} \subseteq$ optimal policies under R_{env}

These results are **general**: they apply to any instantiation of the framework, not just code generation. Dense reward from any grounded source eliminates gradient dead zones; calibration to any verifier prevents false optima.

Implementation Plan

Framework: verl + FSDP2 + vLLM

Models: Qwen3-4B-Instruct (primary) · Qwen3-14B-Instruct (scale) · OLMo3-7B-Instruct (secondary)

Two feedback modalities (demonstrating generality):

- **Code:** LeetCode / LiveCodeBench — stack traces, compiler errors
- **Math:** MATH / AIME — interpreter errors, assertion failures

Six baselines:

Method	Reward	Adapts?	Labels?
GRPO (binary)	$R_{\text{env}} \in \{0, 1\}$	N/A	No
Test-case fraction	$R = k/n$	N/A	No
External judge (Haiku)	Dense, frozen	Static	No

E1: Signal Validation (Go/No-Go)

Frozen Qwen3-4B, 5K rollouts, no RL. Teacher AUC under context ablations:

Condition	Context	Expected
(a)	Solution only (ungrounded)	Low
(b)	+ execution feedback f_i	High
(c)	+ reasoning trace r_i	Moderate
(d)	Full $(a + f + r)$	Highest

Gate: If (b) $\approx$ (a) at 4B $\rightarrow$ pivot to 14B or stop.

Run on **both domains** (code + math) to confirm generality of the grounding signal across feedback modalities.

E2: Self-Distilled Reward Effectiveness (Core Result)

Full GRPO training, 200–400 steps.

Hypotheses:

1. Self-distilled reward > binary GRPO on Pass@1 and convergence
2. Matches or exceeds external judge at fraction of cost
3. Grounded > ungrounded self-reward (grounding is essential)
4. Self-judgment > test-case fraction within same pass-fraction bucket (qualitatively different signal)

Across both domains to demonstrate the framework, not just one trick.

Additional: Gradient norm analysis (validate theory), group size ablation (compute efficiency), stratified analysis by difficulty.

E3: Judge Maintenance (Key Novel Experiment)

Track teacher AUC every 20 steps throughout RL.

Condition	Maintenance	Labels?
(a) None	$\lambda_1 = \lambda_2 = 0$	No
(b) $\mathcal{L}_{\text{pred}}$ only	Feedback prediction	No
(c) $\mathcal{L}_{\text{cal}}$ only	Verifier calibration	No
(d) $\mathcal{L}_{\text{pred}} + \mathcal{L}_{\text{cal}}$	Full label-free	No
(e) $\mathcal{L}_{\text{CE}}$	Static labeled buffer	Yes

Hypothesis: (a) degrades; (e) degrades slowly (stale buffer); (d) stays flat.

The maintenance mechanism is **domain-agnostic**: $\mathcal{L}_{\text{pred}}$ applies to any environment with structured feedback.

E4–E6: Supporting Experiments

E4 — Reward hacking dynamics

- Track hack rate over training (external labeler)
- Does the exploit class suppress hacking? Does grounding alone help?
- Self-judge as a reward hacking *detector*, not just reward source

E5 — Scale and domain transfer

- 4B vs 14B self-judgment quality
- Code vs math: different feedback modality, same framework
- Does the framework transfer without modification?

E6 — Mechanistic analysis

- What does the teacher attend to in f_i?
- Empirically estimate $I(Q^*; f \mid R_{\text{env}}, a)$ across both domains

Key Ablations

Ablation	Experiment	Values
Judgment granularity	E1, E2	Binary vs 5-class
λ_1 (feedback pred.)	E3	$\{0.01, 0.1, 0.5, 1.0\}$
λ_2 (calibration)	E3	$\{0.01, 0.1, 0.5, 1.0\}$
Warm-start SFT	E1, E3	With vs without
Group size G	E2	$\{4, 8, 16\}$
w_{exploit}	E4	$\{-1.0, -0.5, -0.25, 0\}$
Feedback modality	E5	Stack traces vs interpreter

Risks and Mitigations

Risk	Mitigation
4B judgment capacity	Warm-start + $\mathcal{L}_{\text{pred}}$; 14B fallback
$\mathcal{L}_{\text{pred}}$ doesn't transfer to judgment	$\mathcal{L}_{\text{cal}}$ bridges; ablate each
Reward collapse	$\mathcal{L}_{\text{pred}}$ forces f_i engagement
Self-judge hacking	$\mathcal{L}_{\text{cal}} + \mathcal{L}_{\text{pred}}$ + consistency checks
Fraction baseline matches	Within-bucket analysis; single-verdict envs
Generality claim	Two domains, two feedback modalities

Timeline

Phase	Weeks	Milestone
Infrastructure (E0)	1	verl pipeline confirmed
Signal validation (E1)	2–3	Go/no-go, both domains
Core experiments (E2+E3)	4–8	Main results
Supporting (E4+E5)	9–11	Scale + hacking + transfer
Mechanistic + writing (E6)	11–13	Paper complete

Target: **NeurIPS 2026**

Summary

Framework:

- Self-distilling rewards from env feedback
- Policy as grounded self-judge
- General: any structured feedback

Instantiation (EGSR):

- RLVR with GRPO
- Code + math domains
- verl + vLLM, Qwen3-4B/14B

Multiple benefits from one mechanism:

- Dense reward (solves sparsity)
- On-policy (no distributional gap)
- Grounded (no reward collapse)
- Hack suppression (exploit class)
- Compute-efficient (smaller G)
- Label-free (no replay buffer)

Novel theory:

- Variance reduction guarantee
- Info-theoretic feedback bound
- Calibration safety

Discussion

1. **Framing:** "Self-distilling rewards from environment feedback" as a general paradigm — RLVR is the cleanest instantiation, not the only one
2. **Generality evidence:** Two domains (code + math) with different feedback modalities. Sufficient for NeurIPS, or need a third?
3. **Scope:** Non-verifiable extension (NLI for summarization) — include as experiment or leave as future work?
4. **Key experiment:** E3 (judge maintenance) is the strongest novel result — frame it as general: self-supervised maintenance works wherever there's structured feedback
5. **Title:** "Self-Distilling Rewards from Environment Feedback" vs "On-Policy Self-Judging Rewards via Environment Feedback Grounding"?